

0:00–1:30 · Вступление и проблема

Здравствуйте! Меня зовут Павел Савельичев. Тема моей дипломной работы — автоматизированный сервис уведомлений на основе событий. Сам сервис называется `alerting-service`.

Коротко о контексте. Проект — платформа онлайн-кинотеатра из набора микросервисов. К началу работы на ней было два независимых конвейера. Первый — **сбор поведения**: события идут из `activity-tracker` через Kafka в аналитическое хранилище StarRocks. Второй — **доставка уведомлений**: `notifications-service` отправляет письма и сообщения в веб-сокет.

Проблема — между конвейерами не хватало связующего звена, превращающего **«данные»** в **«действие»**. Чтобы запустить рассылку по поведению, приходилось вручную вовлекать маркетолога-оператора, и от идеи до отправки проходили дни. Моя работа закрывает этот разрыв.

1:30–4:00 · Суть сервиса

Если одной фразой — это **движок SQL-правил поверх аналитического хранилища**.

Аналитик описывает аудиторию обычным SQL-запросом: пишет `SELECT`, который возвращает идентификаторы пользователей и контекст для персонализации письма. Регистрирует его как правило одной функцией — `adm_create_rule`. У правила есть расписание, шаблон письма и лимит частоты.

Дальше всё автоматически. По расписанию движок делает четыре вещи: **(1)** выполняет SQL в StarRocks и получает список пользователей; **(2)** отсекает попавших под лимит, чтобы не заспамить; **(3)** записывает историю отправок; **(4)** одной транзакцией создаёт задачу на рассылку в `notifications-service`.

Главное изменение: из цикла убран оператор. Аналитик сам запускает поведенческую рассылку, время от идеи до отправки — с дней до часов.

Зачем бизнесу: автоматизация типовых сценариев — возврат угасшего зрителя (win-back); промо при росте интереса в сегменте; рекомендация альтернативы бросившим новинку.

Аналоги — Mindbox, Customer.io, Braze. Моё отличие — **целевой пользователь**. Они делают для маркетолога без SQL и строят сложные визуальные конструкторы. Я делаю инструмент для аналитика, который SQL знает — интерфейс проще, гибкость выше: любая выборка — один `SELECT`.

4:00–6:00 · Стек технологий

Главный принцип — **переиспользовать то, что в платформе уже есть**, и не плодить сервисы. Сервис получился тонким.

Движок — Python 3.12 + планировщик APScheduler. Распределённую очередь (Celery) сознательно не брал: планировщик в том же процессе достаточен, а распределённая очередь уже есть в `notifications-service`.

Аналитическое хранилище — StarRocks, он уже был в проекте как OLAP. Удобен: говорит по протоколу MySQL, поддерживает материализованные представления, сам подгружает события из Kafka механизмом Routine Load.

Справочники (фильмы, пользователи, жанры) синхронизирую из Postgres в StarRocks нативно — внешним JDBC-каталогом и задачей `SUBMIT TASK` по расписанию. Отдельный Python-ETL здесь избыточен: несколько таблиц с полной перезаливкой раз в час.

Правила и историю храню в PostgreSQL — в той же базе, где `notifications`. Это принципиально: благодаря общей базе критичный шаг укладывается в одну транзакцию.

Для дашбордов — Apache Superset: open-source, готовый коннектор к StarRocks, на аналитических объёмах лучше Metabase.

6:00–8:00 · Почему SQL-интерфейс, а не REST

Главное архитектурное решение — интерфейс управления это SQL-функции, а не REST-API.

Пользователь. Целевой пользователь — аналитик, который весь день в DBeaver. Вызвать `SELECT adm_create_rule(...)` ему естественнее, чем формировать HTTP-запрос через `curl`.

Меньше слоёв и кода. Не нужно поднимать HTTP-API и отдельный веб-интерфейс. Всё управление — набор SQL-функций в схеме `alerting`.

Гибкость. Любое условие выборки — один `SELECT` поверх витрин. Не нужен визуальный конструктор, который всегда чего-то не умеет.

От REST не отказываюсь совсем — вынес его в «возможные улучшения». Понадобится — это тонкая обёртка над теми же функциями, без дублирования логики.

8:00–14:00 · Демонстрация

Покажу всё вживую. → Контур: событие → StarRocks → SQL-правило → письмо → клик → событие обратно → воронка отклика.

1. Демо-данные: `seed-users` + `trigger-events winback` → поток `view` в Kafka → `user_events` (Routine Load). 2. Обновить витрины: `REFRESH MATERIALIZED VIEW mv_user_activity / mv_user_top_genres`.

3. Правило (movies-pg): `alerting.adm_create_rule(...)` + `adm_enable_rule`. Показать валидацию — осмысленная ошибка на `DELETE FROM`. 4. Dry-run: `adm_dry_run_rule` → `v_runs`: matched 20, dispatched 0 (писем нет).

5. Боевой запуск: `adm_trigger_rule` → 20 писем в Mailpit, у каждого свой top-3 жанров (per-user context). 6. Лимит: повторный `adm_trigger_rule` → `after_cap` 0, новых писем нет (`per_rule_per_user_days=30`).

7. Замыкание петли. В письме ссылка `GET /ugc/email/click` → событие `recommendation` в Kafka → `user_events`. Журнал копируется в `dispatch_log` (каждые 30с), `mv_rule_conversion` по таймеру (10с). В Superset — Funnel «отправлено → перешли по ссылке». Открыть пару писем, нажать ссылку → через ~10–20с воронка наполняется сама. Повторный клик дубля не создаёт (`request_id = uuid5(rule, run, user)`).

14:00–15:00 · Итоги

В платформу добавилось недостающее звено «данные → действие»: аналитик одним SQL-правилом запускает поведенческую рассылку.

И контур не просто работает, а **измерим**: весь путь от события пользователя до отклика на письмо виден в одной воронке.

На этом всё. Спасибо за внимание — готов ответить на вопросы.

🔁 Если спросят

Почему не Celery? Распределённая очередь уже есть в notifications; APScheduler в том же процессе достаточен — не дублирую инфраструктуру.

Почему не отдельный ETL для справочников? JDBC Catalog + SUBMIT TASK — нативный механизм StarRocks, та же «оркестрация внутри хранилища», что и Routine Load. Python-воркер оправдан при CDC/сложных преобразованиях — здесь избыточен.

Идемпотентность / сбои? Цепочка «выборка → лимит → история → задача» в одной транзакции; ключ задачи `alerting:{rule_id}:{run_id}` детерминирован — повтор после сбоя дублей не даёт. Прерванные запуски восстанавливаются.

Безопасность SQL-правил? Движок читает StarRocks под ролью `alert_reader` (только чтение); некорректный SQL отклоняется с осмысленной ошибкой.

Не опасно ли давать аналитику SQL? Read-only роль + валидация + правило проходит dry-run перед боевым запуском.

Цифры под рукой: Python 3.12 · StarRocks (MySQL-протокол, порт 9030) · Postgres 5438, БД movies · Mailpit :8025 · Superset :8088 · Kafka UI :8081 · win-back: matched 20, лимит 30 дней · витрины 6 (mv_*) · справочники из Postgres раз в 1 час.